

2021

1. a) open system

- i) Interacts with external systems/environment
 - ii) It exchanges data, resources or services.
 - iii) It is usually connected to networks or APIs
 - iv) The system is more flexible and extensible
 - v) security risks are higher because of external access.
- ex → web apps, cloud system, online banking.

Close system

- i) works independently with little or no interaction with external environment.
 - ii) It does minimal or no data exchange.
 - iii) This type of system is completely isolated.
 - iv) These are less flexible.
 - v) Lower security risks.
- ex → offline calculator, standalone embedded software.

b) In software engineering a software fault is a defect, mistake or flaw in a software program that may cause the system to produce incorrect results or behave unexpectedly.

ex
 $2+2=4$ (expected)
 Program outputs (5). [Fault]

Types of faults

1. Syntax fault → Grammatical coding mistake
2. Logical fault → program runs but gives wrong output
3. Runtime fault → Error during execution
4. Interface fault → prob in communication between modules.

Cause of faults

- ⊙ Incorrect Requirements.
- ⊙ Poor design
- ⊙ Coding mistake
- ⊙ Communication error
- ⊙ Lack of testing
- ⊙ Human errors.

Fault → Error
 ↓
 Failure.

c) In the context of software ~~engineer~~ development, "V and V" stands for Verification and Validation. It is an essential set of activities performed throughout the SDLC to ensure that the software product meets the intended requirements, functions correctly and is of high quality.

Verification activities →

- Reviews
- Inspections
- Walkthroughs
- Static testing

Validation activities →

- Functional testing
- Systems "
- User acceptance testing.

Roles in SDLC →

1. Detects errors early.
2. Improves software reliability.
3. Ensure software meets the requirements.
4. Reduces development costs.
5. Increases customer satisfaction.
6. Prevents software failures.

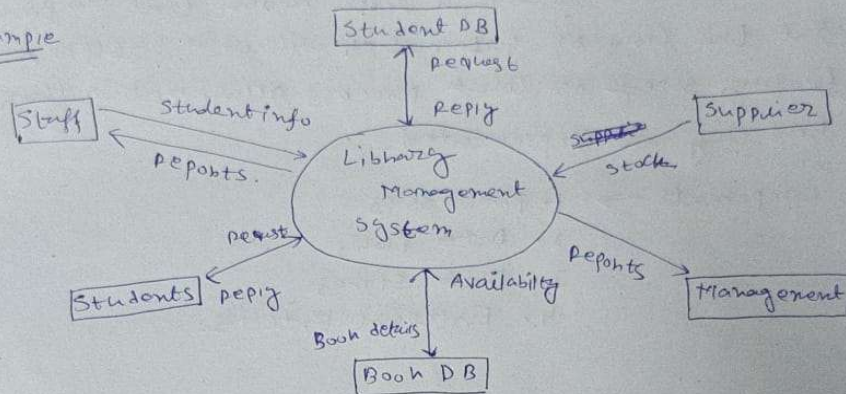
d) The purpose of unit testing is to validate that the smallest testable parts of software (units) function correctly and as intended in isolation. By isolating individual functions, methods or classes devs can ensure the code logic of each component is sound before integrating it into larger system.

Reasons to do it →

- Detects errors early.
- Ensure correctness of individual modules.
- Simplifies debugging.
- Improves SW quality.
- Supports integration testing.
- Reduces dev cost.

e) A Context diagram is a high-level visual representation that shows how a system interacts with external entities like users, other systems or orgs. Also known as a level 0 DFD, it defines the system's boundaries and maps the flow of data in and out without showing internal details.

example



It helps the non-technical stakeholders understand the system's role without getting lost in technical details, as it is simple [Level 0] it avoids early complex modelling, highlighting interactions with external environments, which is necessary to identify functional requirements.

F) Low coupling is desirable because →

1. Easy maintenance → A change in one module does not require change in others.
2. Better flexibility → Modules can be modified or replaced independently.
3. Improved reusability → Independent modules can be reused in other projects.
4. simpler testing
5. reduced complexity
6. Fewer errors propagate
7. supports parallel development.

ex If a payment module directly depends heavily on the order module, changing the order module may break it but if the module communicates through API/Interfaces, one can change without affecting other.

g) A logical DFD is a high level model that maps out the flow of information within a system, focusing solely on what process occurs rather than how they are implemented.

components →

1. processes
2. Data flows
3. Data Stores
4. External Entities.

h) A stub simulates the behaviour of a lower-level ~~node~~ module that has not been developed yet or is unavailable. It performs limited functionality

like →

- Accepting input
- Returning predefined output.
- Displaying messages
- Imitating module behaviour.

ex main module calls the payment module which is not been developed yet. A stub may temporarily return "payment successful".

2.a. What are the advantages of the Prototype Model? Regardless of its merits, why is it a costly model to emulate?

The **Prototype Model** is a software development model in which a preliminary working version (prototype) of the software is built, tested, and refined based on user feedback before developing the final system.

Advantages

1. Better Understanding of Requirements

- Users can see a working model and clarify their needs more easily.

2. Early User Involvement

- Continuous interaction between developers and users improves communication.

3. Reduces Requirement Errors

- Missing or unclear requirements can be identified early.

4. Improves User Satisfaction

- Users get a system closer to their expectations.

5. Early Detection of Problems

- Design and functionality issues are found before full development.

6. Flexible and Easy to Modify

- Changes can be incorporated during development.

7. Useful for Complex Systems

- Helpful when requirements are not clearly defined initially.

Despite its advantages, the Prototype Model can become expensive for several reasons:

Reasons

1. Repeated Modifications

- Continuous changes based on user feedback increase development effort and cost.

2. Extra Development Time

- Building prototypes before the final product requires additional time.

3. Higher Resource Consumption

- More manpower, tools, and testing efforts are needed.

4. Frequent User Interaction

- Constant meetings and evaluations increase overall project cost.

5. Possibility of Scope Expansion

- Users may keep demanding new features after seeing prototypes.

6. Wastage of Prototype Code

- Some prototype components may not be reusable in the final system.

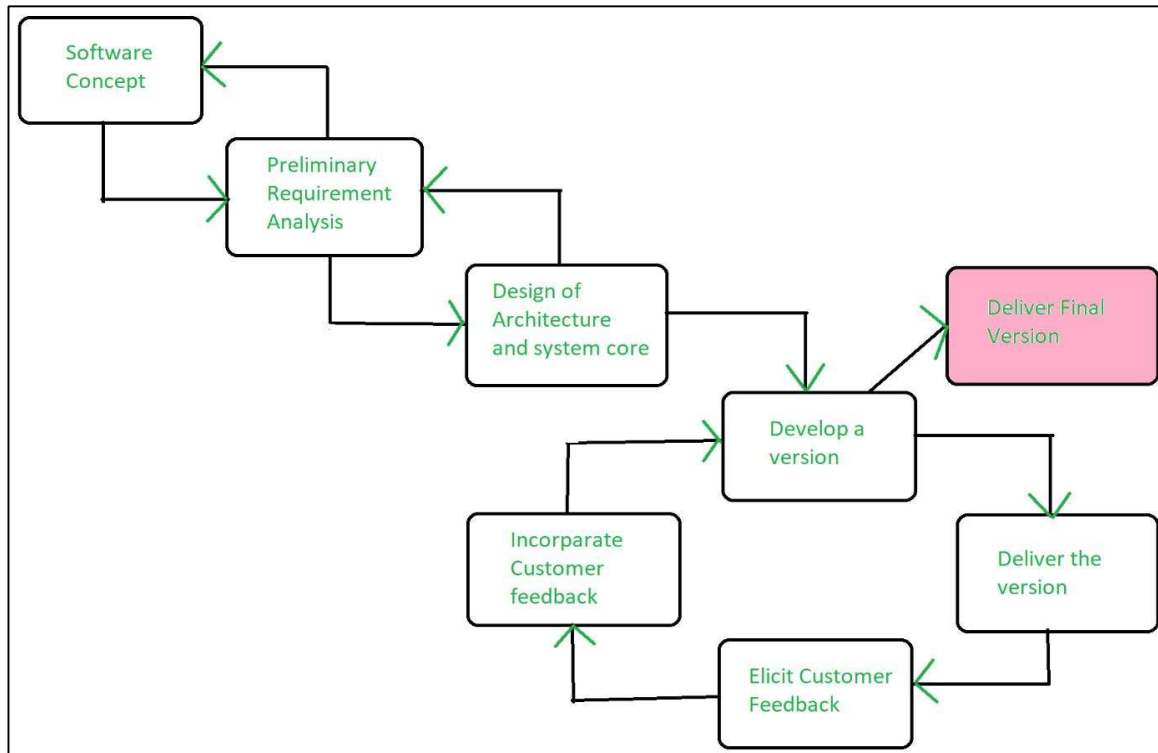
7. Complex Management

- Managing iterative changes and versions becomes difficult.

b. Discuss the Evolutionary Model in brief, along with its suitable domain of application.

The Evolutionary development model divides the development cycle into smaller, incremental waterfall models in which users can get access to the product at the end of each cycle.

1. Feedback is provided by the users on the product for the planning stage of the next cycle and the development team responds, often by changing the product, plan, or process.
2. Therefore, the software product evolves with time.
3. All the models have the disadvantage that the duration of time from the start of the project to the delivery time of a solution is very high.
4. The evolutionary model solves this problem with a different approach.
5. The evolutionary model suggests breaking down work into smaller chunks, prioritizing them, and then delivering those chunks to the customer one by one.
6. The number of chunks is huge and is the number of deliveries made to the customer.
7. The main advantage is that the customer's confidence increases as he constantly gets quantifiable goods or services from the beginning of the project to verify and validate his requirements.
8. The model allows for changing requirements as well as all work is broken down into maintainable work chunks.



Characteristics

- Incremental development
- Iterative process
- Continuous user involvement
- Flexible to requirement changes

Advantages

1. Handles changing requirements easily.
2. Early delivery of working software.
3. Reduces project risk.
4. Improves customer satisfaction.
5. Easier debugging and testing.

Disadvantages

1. Requires skilled management.
2. Development cost may increase.
3. Documentation may become difficult.
4. Continuous changes may affect design quality.

The Evolutionary Model is suitable for projects where:

- requirements are unclear or changing,
- early delivery is important,
- continuous customer feedback is needed.

Suitable Applications

- Web applications
- E-commerce systems
- Banking software
- Mobile applications
- AI and research-based systems
- Large business management systems

3.a What is the role of an SRS? What are the components of a good SRS?

SRS stands for Software Requirement Specification. It is one of the most important documents in Software Engineering. An SRS describes all the functional and non-functional requirements of a software system in a clear and organized manner before development begins.

It acts as an agreement between the customer and the development team regarding what the software should do.

Role of an SRS

The Software Requirement Specification plays a vital role in the Software Development Life Cycle (SDLC). Its major roles are discussed below.

1. Acts as a Communication Medium

An SRS serves as a bridge between clients, developers, testers, and project managers. It helps all stakeholders understand the system requirements clearly and reduces misunderstanding.

2. Basis for Software Design and Development

Developers use the SRS as the foundation for designing and coding the software system. All development activities are performed according to the specifications mentioned in the document.

3. Helps in Testing and Validation

Testers prepare test cases based on the requirements defined in the SRS. It helps verify whether the developed software satisfies customer requirements

4. Reduces Development Errors

A properly prepared SRS identifies requirements clearly at an early stage, reducing ambiguity and minimizing costly errors during development.

5. Supports Project Planning

Project managers use the SRS for estimating cost, time, manpower, and resources required for the project.

6. Acts as a Reference Document

During maintenance and future upgrades, the SRS works as an important reference document for understanding the system.

Components of a Good SRS

A good SRS should be complete, clear, and well-structured. The important components are as follows.

1. Introduction

This section contains:

- purpose of the software,
- scope of the project,
- definitions and abbreviations,
- references and overview.

It provides a general understanding of the system.

2. Overall Description

This section describes:

- product perspective,
- product functions,
- user characteristics,
- assumptions and constraints.

It explains the general behaviour and environment of the system.

3. Functional Requirements

These specify the functions and services the software must provide.

Examples:

- login functionality,
- payment processing,

- report generation.

This is the core part of the SRS.

4. Non-Functional Requirements

These define quality attributes of the system such as:

- performance,
- security,
- reliability,
- usability,
- maintainability.

They describe how the system should perform.

5. External Interface Requirements

This section specifies interfaces with:

- users,
- hardware,
- software,
- communication systems.

Examples include GUI design and API interactions.

6. Data Requirements

It includes:

- database requirements,
- data structures,
- input and output formats.

7. Constraints and Assumptions

This section mentions limitations and assumptions related to:

- hardware,
- software,
- legal requirements,
- standards.

Characteristics of a Good SRS

A good SRS should be:

1. **Correctness:**

User review is used to ensure the correctness of requirements stated in the SRS. SRS is said to be correct if it covers all the requirements that are actually expected from the system.

2. **Completeness:**

Completeness of SRS indicates every sense of completion including the numbering of all the pages, resolving the to be determined parts to as much extent as possible as well as covering all the functional and non-functional requirements properly.

3. **Consistency:**

Requirements in SRS are said to be consistent if there are no conflicts between any set of requirements. Examples of conflict include differences in terminologies used at separate places, logical conflicts like time period of report generation, etc.

4. **Unambiguousness:**

A SRS is said to be unambiguous if all the requirements stated have only 1 interpretation. Some of the ways to prevent unambiguousness include the use of modelling techniques like ER diagrams, proper reviews and buddy checks, etc.

5. **Ranking for importance and stability:**

There should be a criterion to classify the requirements as less or more important or more specifically as desirable or essential. An identifier mark can be used with every requirement to indicate its rank or stability.

6. **Modifiability:**

SRS should be made as modifiable as possible and should be capable of easily accepting changes to the system to some extent. Modifications should be properly indexed and cross-referenced.

7. **Verifiability:**

A SRS is verifiable if there exists a specific technique to quantifiably measure the extent to which every requirement is met by the system. For example, a requirement stating that the system must be user-friendly is not verifiable and listing such requirements should be avoided.

8. **Traceability:**

One should be able to trace a requirement to design component and then to code segment in the program. Similarly, one should be able to trace a requirement to the corresponding test cases.

b. Discuss the different types of 'coupling' in detail.

In Software Engineering, **coupling** refers to the degree of dependency between different modules of a software system. It indicates how strongly one module is connected to another.

If modules are highly dependent on each other, the system is said to have **high coupling**. If modules work mostly independently, the system has **low coupling**.

Low coupling is desirable because it improves maintainability, flexibility, testing, and reusability of software.

Types of Coupling

The different types of coupling are discussed below in order from **highest (worst)** to **lowest (best)** coupling.

1. Content Coupling

Content coupling occurs when one module directly accesses or modifies the internal data or code of another module.

This is the highest and worst form of coupling because changes in one module can heavily affect another module.

Characteristics

- One module depends on internal implementation of another.
- Violates information hiding.
- Difficult to maintain.

Example

A module directly modifies local variables of another module.

Disadvantages

- Very difficult debugging
- Poor maintainability
- High dependency

2. Common Coupling

Common coupling occurs when multiple modules share the same global data.

If one module changes the shared data, all other dependent modules may be affected.

Characteristics

- Uses global variables.
- Changes are difficult to track.

Example

Several functions accessing the same global variable.

Disadvantages

- Risk of unexpected side effects
- Difficult testing and debugging

3. External Coupling

External coupling occurs when modules depend on externally imposed data formats, communication protocols, or device interfaces.

Characteristics

- Dependency on external systems or files.
- Modules interact through externally defined formats.

Example

Two modules communicating through a shared communication protocol.

Disadvantages

- Changes in external interface affect multiple modules.

4. Control Coupling

Control coupling occurs when one module controls the behavior of another module by passing control information such as flags or commands.

Characteristics

- One module influences the logic of another.
- Reduces module independence.

Example

Passing a flag variable that decides what operation another module performs.

```
process(data, flag)
```

Disadvantages

- Makes code complex
- Harder to understand and modify

5. Stamp Coupling (Data-Structured Coupling)

Stamp coupling occurs when modules share a composite data structure, but the called module uses only a part of it.

Characteristics

- Whole records or structures are passed.
- Some unnecessary data is transferred.

Example

Passing an entire student record when only student ID is needed.

Disadvantages

- Increased dependency on data structure format

6. Data Coupling

Data coupling occurs when modules interact by passing only the necessary data items as parameters.

This is considered a good form of coupling.

Characteristics

- Modules communicate through simple data.
- Minimal dependency.

Example

Passing two numbers to an addition function.

`add(a, b)`

Advantages

- Easy maintenance
- Better readability
- Easier testing

7. Message Coupling

Message coupling is the lowest and best form of coupling. Modules communicate only through messages or public interfaces without sharing internal details.

It is commonly used in object-oriented systems.

Characteristics

- Completely independent modules
- Communication through interfaces/messages only

Example

Objects communicating through method calls.

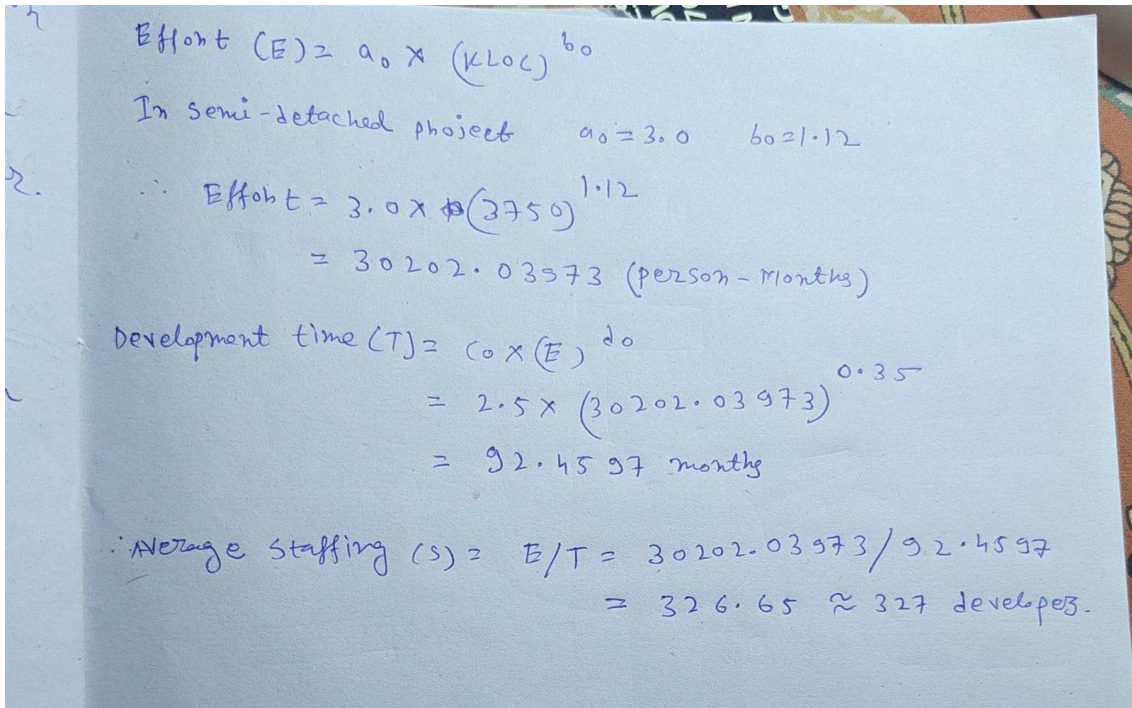
Advantages

- Very high modularity
- Easy reuse and maintenance
- Flexible design

4.a. A software project of type Semi-detached comprises of 3750 KLOC. Compute the development effort using Basic COCOMO

Metric	Formula	Unit of Measurement
Effort (E)	$E = a_b \times (KLOC)^{b_b}$	Person-Months (PM)
Development Time (T)	$T = c_b \times E^{d_b}$	Months (M)
Average Staffing (S)	$S = E / T$	Persons (Headcount)

Project Type	a_b	b_b	c_b	d_b
Organic (Simple, flexible, experienced team)	2.4	1.05	2.5	0.38
Semi-detached (Medium complexity, mixed team)	3.0	1.12	2.5	0.35
Embedded (Highly complex, strict constraints)	3.6	1.20	2.5	0.32



Handwritten calculations for COCOMO model:

$$\text{Effort (E)} = a_b \times (KLOC)^{b_b}$$

In semi-detached project $a_b = 3.0$ $b_b = 1.12$

$$\therefore \text{Effort} = 3.0 \times (3750)^{1.12}$$
$$= 30202.03973 \text{ (person-months)}$$
$$\text{Development time (T)} = c_b \times (E)^{d_b}$$
$$= 2.5 \times (30202.03973)^{0.35}$$
$$= 92.4597 \text{ months}$$
$$\therefore \text{Average Staffing (S)} = E/T = 30202.03973 / 92.4597$$
$$= 326.65 \approx 327 \text{ developers.}$$

b. Describe the features of organic type of software as specified in COCOMO

COCOMO (Constructive Cost Model) is a software cost estimation model proposed by Barry Boehm.

It is used to estimate the effort, development time, and cost required for software projects.

In the Basic COCOMO model, software projects are classified into three categories:

1. Organic
2. Semi-Detached
3. Embedded

The **Organic type** represents small and relatively simple software projects developed by experienced teams in a familiar environment.

Features of Organic Type Software

1. Small Project Size

Organic software projects are generally small in size and complexity. The project usually contains a limited number of modules and functionalities.

Example

- Simple payroll system
- Library management system
- Student record management system

2. Simple and Well-Understood Requirements

The requirements of the software are clear, stable, and well understood from the beginning of development.

There are very few uncertainties or frequent requirement changes.

3. Experienced Development Team

The project is developed by a small team of experienced programmers who are familiar with the application domain and development tools.

Because of their expertise, development becomes easier and faster.

4. Familiar and Stable Environment

Organic projects are developed in a familiar working environment using known technologies and standard development practices.

There are minimal hardware or software constraints.

5. Low Complexity

The software system has low technical and operational complexity.

Algorithms, interfaces, and processing requirements are usually straightforward.

6. Flexible Development Process

Developers have greater flexibility in design and implementation decisions because strict regulations and constraints are generally absent.

7. Less Stringent Reliability Requirements

Organic software usually does not require extremely high reliability or real-time performance.

Minor failures generally do not lead to severe consequences.

8. Short Development Time

Since the project is smaller and simpler, the development cycle is comparatively short.

c. Write the advantages of decision tree in problem design

A **decision tree** is a graphical representation used in problem solving and software design to show different decisions and their possible outcomes in a tree-like structure. It helps designers analyze complex logical conditions systematically.

Decision trees are widely used in software engineering, algorithm design, business analysis, and decision-making processes.

Advantages of Decision Tree

1. Easy to Understand

Decision trees present problems in a simple graphical form.

The branching structure makes the flow of decisions easy to understand even for non-technical users.

2. Helps in Analyzing Complex Logic

Complex conditions and multiple decision paths can be represented clearly using branches and nodes.

This reduces confusion during problem analysis and system design.

3. Improves Decision Making

Decision trees help identify all possible outcomes of a decision.

This allows designers and analysts to select the most appropriate solution.

4. Provides Clear Visualization

The graphical representation gives a clear picture of:

- conditions,

- alternatives,
- actions,
- outcomes.

This improves communication among developers, analysts, and users.

5. Simplifies Problem Decomposition

Large problems can be divided into smaller subproblems using branches in the tree.

This makes problem-solving more organized and manageable.

6. Useful in Program Design

Decision trees help programmers convert logical conditions into program structures such as:

- if-else statements,
- switch cases,
- flow control structures.

7. Helps in Error Detection

Since all decision paths are explicitly shown, missing conditions or logical inconsistencies can be identified easily.

8. Facilitates Testing

Decision trees help testers identify all possible execution paths and prepare test cases effectively.

9. Supports Documentation

A decision tree acts as a useful documentation tool for future reference and maintenance.

5.a. Why is Black Box testing also known as functional testing? How is it different from traditional White Box testing?

Software testing is an important activity in Software Engineering used to detect errors and verify software quality. Two major testing techniques are:

1. Black Box Testing
2. White Box Testing

Black Box testing is also called **functional testing** because it focuses mainly on checking whether the functions of the software work according to specified requirements.

Why is Black Box Testing Known as Functional Testing?

In Black Box Testing, the internal structure or source code of the software is not examined. The tester only checks the input and output behaviour of the system.

The main objective is to verify whether the software functions correctly according to user requirements and specifications.

Thus, Black Box testing is called **functional testing** because it tests the functional behaviour of the software.

Characteristics

- Tests software functionality
- No knowledge of internal code is required
- Focuses on input-output behaviour
- Based on requirements and specifications

Example

Suppose a login page accepts:

- correct username and password → login successful
- wrong password → error message displayed

The tester only checks whether the function behaves correctly, without examining the program code.

Parameters	Black Box Testing	White Box Testing
Definition	Black Box Testing is a way of software testing in which the internal structure or the program or the code is hidden and nothing is known about it.	White Box Testing is a way of testing the software in which the tester has knowledge about the internal structure or the code or the program of the software.
Testing objectives	Black box testing is mainly focused on testing the functionality of the software, ensuring that it meets the requirements and specifications.	White box testing is mainly focused on ensuring that the internal code of the software is correct and efficient.
Testing methods	Black box testing uses methods like Equivalence	White box testing uses methods like Control Flow

	Partitioning , Boundary Value Analysis , and Error Guessing to create test cases.	Testing , Data Flow Testing and Statement Coverage Testing .
Knowledge level	Black box testing does not require any knowledge of the internal workings of the software, and can be performed by testers who are not familiar with programming languages.	White box testing requires knowledge of programming languages, software architecture and design patterns.
Scope	Black box testing is generally used for testing the software at the functional level.	White box testing is used for testing the software at the unit level, integration level and system level.
Implementation	Implementation of code is not needed for black box testing.	Code implementation is necessary for white box testing.
Done By	Black Box Testing is mostly done by software testers.	White Box Testing is mostly done by software developers.
Terminology	Black Box Testing can be referred to as outer or external software testing.	White Box Testing is the inner or the internal software testing.
Testing Level	Black Box Testing is a functional test of the software.	White Box Testing is a structural test of the software.
Testing Initiation	Black Box testing can be initiated based on the requirement specifications document.	White Box testing of software is started after a detail design document.
Programming	No knowledge of programming is required.	It is mandatory to have knowledge of programming.
Testing Focus	Black Box Testing is the behaviour testing of the software.	White Box Testing is the logic testing of the software.
Applicability	Black Box Testing is applicable to the higher levels of testing of software.	White Box Testing is generally applicable to the lower levels of software testing.
Alternative Names	Black Box Testing is also called closed testing.	White Box Testing is also called as clear box testing.

Time Consumption	Black Box Testing is least time consuming.	White Box Testing is most time consuming.
Suitable for Algorithm Testing	Black Box Testing is not suitable or preferred for algorithm testing.	White Box Testing is suitable for algorithm testing.
Approach	Can be done by trial and error ways and methods.	Data domains along with inner or internal boundaries can be better tested.
Example	Search something on google by using keywords	By input to check and verify loops
Exhaustiveness	It is less exhaustive as compared to white box testing.	It is comparatively more exhaustive than black box testing.

b. Design an equivalence class partitioning for a program that accepts as an input, an OTP of 6-digits, and considers all other types of inputs as invalid.

Equivalence Class Partitioning (ECP) is a Black Box testing technique in which input data is divided into groups called equivalence classes.

Each class represents a set of inputs that are expected to be processed similarly by the software.

Instead of testing every possible input, one representative value from each class is tested.

Designing Equivalence Classes

The input conditions are:

1. OTP must contain exactly 6 digits.
2. Only numeric digits are allowed.

Based on these conditions, equivalence classes can be formed as follows.

Valid Equivalence Class

Class	Description	Example
V1	Exactly 6 numeric digits	123456

This class should be accepted by the system.

Invalid Equivalence Classes

Class	Description	Example
I1	Less than 6 digits	12345
I2	More than 6 digits	1234567
I3	Contains alphabets	12AB56
I4	Contains special characters	12@#56
I5	Empty input	
I6	Contains spaces	12 456
I7	Combination of digits and letters	123A56

These classes should be rejected by the system.

Test Cases Based on Equivalence Classes

Test Case	Input	Expected Result
TC1	123456	Valid OTP
TC2	12345	Invalid
TC3	1234567	Invalid
TC4	12AB56	Invalid
TC5	12@#56	Invalid
TC6	Empty input	Invalid
TC7	12 456	Invalid

c. How is testing related to software quality?

Testing is the practical process of evaluating software to ensure it meets requirements, functions correctly, and is reliable, secure, and user-friendly. It directly drives software quality by identifying defects (bugs) before release, reducing costs, validating performance, and ensuring a positive user experience.

Core Relationships Between Testing and Quality:

- **Verification and Validation:** Testing verifies that the product adheres to specifications (built correctly) and validates that it meets user needs (built the right product).
- **Defect Detection & Prevention:** Testing is a proactive approach to finding issues before they impact users. Early testing, such as unit or integration testing, reduces the cost of fixing errors later in development.

- **Evaluation of Quality Standards:** Testing measures software against key quality attributes like functionality, usability, reliability, security, maintainability, and portability.
- **Reduced Risk:** Rigorous testing minimizes the chance of system failure upon deployment, enhancing overall reliability.
- **Improved User Experience (UX):** Testing ensures the software is intuitive and works as expected by the end-user

6.b. Explain the functionality of physical DFD.

A **Physical Data Flow Diagram (Physical DFD)** is a type of Data Flow Diagram that describes how a system is actually implemented in practice.

It shows the physical aspects of the system such as:

- hardware,
- software,
- files,
- databases,
- people,
- input/output devices,
- and methods of data processing.

Unlike a Logical DFD, which focuses on *what the system does*, a Physical DFD focuses on how the system operates and is implemented.

Functionality of Physical DFD

The Physical DFD performs several important functions in software system design and analysis.

1. Shows Actual Implementation of the System

A Physical DFD explains how the system is physically implemented.

It shows:

- where processing occurs,
- who performs the processing,
- what devices or software are used.

This helps developers understand the practical working structure of the system.

2. Identifies Hardware and Software Components

The diagram specifies the physical resources used in the system such as:

- computers,

- servers,
- databases,
- printers,
- communication devices.

This helps in system planning and infrastructure design.

3. Describes Data Movement

A Physical DFD shows how data physically moves through the system.

It includes:

- manual data flow,
- electronic transfer,
- file storage,
- database communication.

Thus, it helps visualize real data processing activities.

4. Defines Manual and Automated Processes

The Physical DFD clearly distinguishes between:

- manual operations performed by people,
- automated processes performed by computers.

This helps organizations understand task distribution.

5. Assists in System Design and Implementation

Developers use the Physical DFD during coding and implementation because it provides technical details about:

- program modules,
- storage systems,
- interfaces,
- processing methods.

6. Helps in Performance Analysis

Since the actual implementation is represented, Physical DFDs help identify:

- bottlenecks,
- redundant processes,
- inefficient data movement.

This improves system optimization.

7. Supports Documentation and Maintenance

The Physical DFD serves as an important technical document for future system maintenance and upgrades.

New developers can understand the existing implementation more easily.

Example

In a Library Management System, a Physical DFD may show:

- barcode scanner for book entry,
- database server storing records,
- librarian entering student details through a computer terminal,
- printed issue receipts.

These implementation details are not shown in a Logical DFD.

7.a. Briefly discuss different Software Quality Assurance (SQA) activities in brief.

Software Quality Assurance (SQA) refers to a set of activities used to ensure that software processes and products meet specified quality standards and user requirements.

The main objective of SQA is to prevent defects, improve software quality, and ensure reliability throughout the Software Development Life Cycle (SDLC).

SQA activities are performed at different stages of software development to monitor, evaluate, and improve both the development process and the final product

- **SQA Management Plan:** Make a plan for how you will carry out the SQA throughout the project. Think about which set of software engineering activities are the best for project. check level of SQA team skills.
- **Set The Check Points:** SQA team should set checkpoints. Evaluate the performance of the project on the basis of collected data on different check points.
- **Measure Change Impact:** The changes for making the correction of an error sometimes re introduces more errors keep the measure of impact of change on project. Reset the new change to check the compatibility of this fix with whole project.
- **Multi testing Strategy:** Do not depend on a single testing approach. When you have a lot of testing approaches available use them.

- **Manage Good Relations:** In the working environment managing good relations with other teams involved in the project development is mandatory. Bad relation of SQA team with programmers team will impact directly and badly on project.
- **Maintaining records and reports:** Comprehensively document and share all QA records, including test cases, defects, changes, and cycles, for stakeholder awareness and future reference.
- **Reviews software engineering activities:** The SQA group identifies and documents the processes. The group also verifies the correctness of software product.
- **Formalize deviation handling:** Track and document software deviations meticulously. Follows established procedures for handling variances

b. How are the terms 'reliability' and 'maintainability' related to software quality

Reliability and Software Quality

Reliability is the ability of software to perform its required functions correctly and continuously without failure for a specified period of time. It is an important attribute of software quality because high-quality software should produce accurate results and operate consistently under different conditions. Reliable software reduces system crashes, improves user confidence, and ensures stable performance. For example, banking software must reliably process transactions without errors.

Maintainability and Software Quality

Maintainability is the ease with which software can be modified, corrected, updated, or improved after deployment. It is related to software quality because good-quality software should be easy to maintain and enhance. Highly maintainable software reduces maintenance cost and effort, supports future updates, and increases software life. Proper documentation, modular design, and low coupling improve maintainability.

8.a. Briefly discuss different stages of SDLC.

The Software Development Life Cycle (SDLC) is a structured process used to plan, design, develop, test, deploy, and maintain software. It ensures a systematic workflow and helps align software development with business goals and user requirements.

- Provides a clear and organized framework for managing development phases
- Helps in early detection of defects, reducing overall cost and time
- Ensures high-quality software delivery that meets user expectations

Stages of the Software Development Life Cycle

The Software Development Life Cycle (SDLC) typically consists of six or seven stages, depending on the development model used.

Stage 1: Planning & Feasibility Analysis

This stage determines whether the project is technically, financially, and operationally feasible.

- **Activities:** Feasibility analysis, cost estimation, scheduling, resource planning
- **Output:** Project Plan, Feasibility Report
- **Key Roles:** Project Managers, Senior Engineers, Stakeholders

Stage 2: Requirement Specification (SRS)

In this stage, detailed functional and non-functional requirements are documented clearly and approved by stakeholders.

- **Activities:** Requirement gathering, validation, documentation
- **Output:** Software Requirement Specification (SRS)
- **Key Roles:** Business Analysts, Product Owners

Stage 3: System Design

In this stage, the approved requirements are transformed into a technical blueprint for implementation.

- **High-Level Design (HLD):** Defines system architecture, technology stack, database design, and major modules.
- **Low-Level Design (LLD):** Specifies component logic, APIs, data structures, and workflows.
- **Output:** Design Document Specification (DDS)

Stage 4: Development (Coding)

Developers build the software based on the approved design.

- **Activities:** Coding, code reviews, unit testing, version control management
- **Tools:** IDEs, version control systems, debuggers
- **Output:** Source code, executable application
- **Key Roles:** Frontend, Backend, Full Stack Developers

Stage 5: Testing

Testing ensures the software meets requirements and is free from defects before release.

Types of Testing Include:

- **Unit Testing:** Verifies individual components

- **Integration Testing:** Ensures modules work together
- **System Testing:** Validates the complete system
- **User Acceptance Testing (UAT):** Confirms business requirements are met
- **Output:** Test cases, defect reports, quality metrics

Stage 6: Deployment

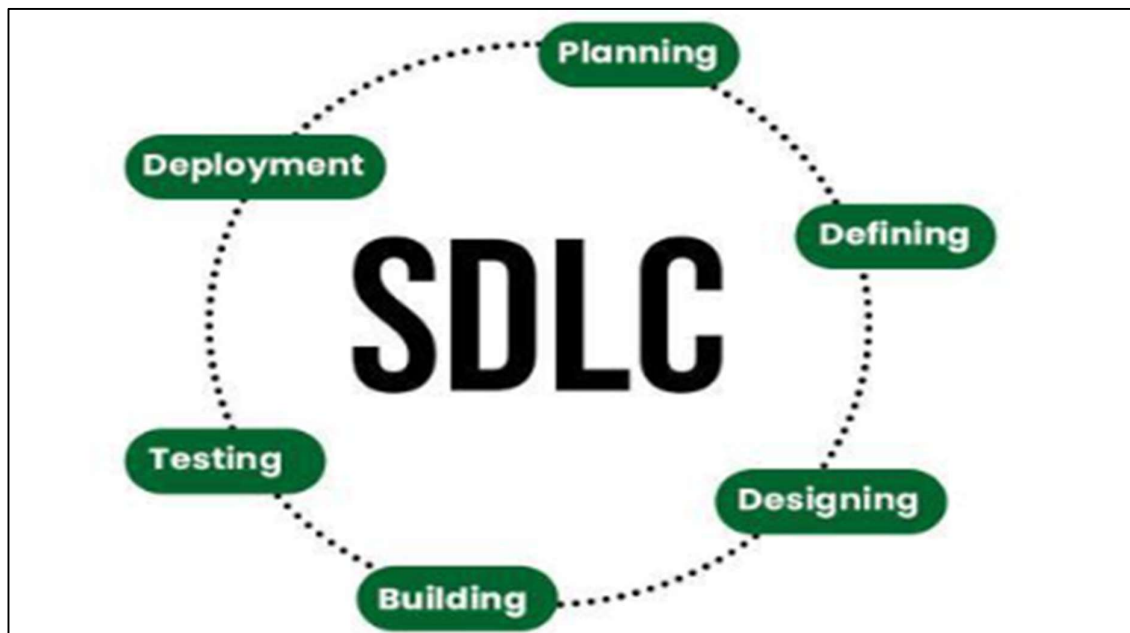
The tested software is released to users.

- **Activities:** Production setup, deployment, smoke testing
- **Modern Approach:** Continuous Integration and Continuous Deployment (CI/CD) pipelines for faster and reliable releases
- **Output:** Live application
- **Key Roles:** DevOps Engineers, Release Managers

Stage 7: Maintenance

Post-deployment support ensures long-term usability.

- **Activities:** Bug fixes, performance tuning, updates, feature enhancements
- **Output:** Patches, updates, new versions
- **Key Roles:** Support Engineers, Developers



b. Define : Computer based systems engineering

Computer-Based Systems Engineering (CBSE) is the systematic approach of designing, developing, integrating, and managing computer-based systems that combine hardware, software, data, procedures, and users to perform specific tasks efficiently.

Explanation

A computer-based system consists of various interconnected components such as:

- hardware,
- software,
- databases,
- networks,
- and human users.

Computer-Based Systems Engineering focuses on analyzing system requirements and developing an integrated system that functions correctly and efficiently.

It applies engineering principles and software engineering techniques to build reliable and effective computer-based solutions.

Objectives

- Improve system efficiency
- Ensure reliability and accuracy
- Integrate hardware and software properly
- Satisfy user requirements
- Support automation and decision-making

Examples

- Banking systems
- Railway reservation systems
- Hospital management systems
- Air traffic control systems
- E-commerce platforms